Basic Electronics & Embedded Systems – Q&A Guide

1. What is the difference between Micro-controller and Microprocessor?

Feature	Micro-controller	Microprocessor
Definition	A compact IC designed to perform specific tasks.	A powerful IC used for general-purpose computing.
Components	Includes CPU, RAM, ROM, I/O ports, and timers in one chip.	Only the CPU; needs external RAM, ROM, I/O, etc.
Used in	Embedded systems (e.g., washing machines, robots).	Personal computers, laptops, etc.
Cost & Power	Low cost, low power consumption.	Higher cost and power consumption.

2. What is the difference between TRIS and PORT Register?

Register	Description
TRIS (Tri-State)	Used to **set direction** of the pin: `1` for input, `0` for output.
PORT	Used to **read or write** data to the pin depending on its direction.

Example:

```
TRISB = 0x00; // Set PORTB as output
PORTB = 0xFF; // Set all PORTB pins HIGH
```

3. What is a Register in Micro-Controller

A **register** is a small, fast memory location inside the microcontroller used to store temporary data and control the behavior of hardware.

Types:

- Data Registers (e.g., PORTA, PORTB)
- Control Registers (e.g., TRIS, ADCON)
- Status Registers (e.g., flags)

4. Write LED Blink Program with 5-Second Time Interval (PIC16F877A)

```c
#include <xc.h>
#define _XTAL_FREQ 20000000

void main() {
    TRISD = 0x00; // Set PORTD as output
    while(1) {
        PORTD = 0xFF; // Turn ON LEDs
        __delay_ms(5000); // Wait 5 seconds
        PORTD = 0x00; // Turn OFF LEDs
        __delay_ms(5000); // Wait 5 seconds
    }
}
```

---

5. What is the difference between while loop and do-while loop?

Feature	`while` loop	`do-while` loop
Condition Check	Before execution	After execution
Executes at least once?	No	Yes

Syntax	`while(condition)`
	`{`
	`//Program to run`
	`}`
		`Do {`
		`// program to run`
		`} while(condition);`

6. What is a Resistor and Explain Resistor Colour Coding

A **resistor** is an electronic component that limits the flow of current in a circuit.

Resistor Colour Code (4-band):

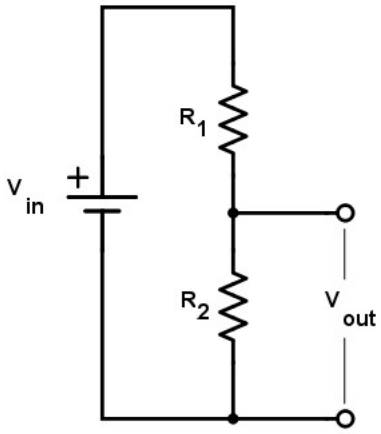
- 1st Band = 1st Digit
- 2nd Band = 2nd Digit
- 3rd Band = Multiplier
- 4th Band = Tolerance

Color	Color	1st Band	2nd Band	3rd Band Multiplier	4th Band Tolerance
Black		0	0	x1Ω	
Brown		1	1	x10Ω	±1%
Red		2	2	x100Ω	±2%
Orange		3	3	x1kΩ	
Yellow		4	4	x10kΩ	
Green		5	5	x100kΩ	±0.5%
Blue		6	6	x1MΩ	±0.25%
Violet		7	7	x10MΩ	±0.10%
Grey		8	8	x100MΩ	±0.05%
White		9	9	x1GΩ	
Gold				x0.1Ω	±5%
Silver				x0.01Ω	±10%

Example:
Red, Violet, Yellow, Gold = 270kΩ ±5%

7. Explain Voltage Divider Circuit Using Resistors

A **voltage divider** is a simple circuit used to reduce voltage using two resistors in series.



Formula:

$$V_{out} = V_{in} \times \frac{R2}{(R1 + R2)}$$

Where:

Vout = Output voltage. This is the scaled down voltage.

Vin = Input voltage.

R1, R2 = Resistor values. The ratio (R2) / (R1 + R2) determines the scale factor.

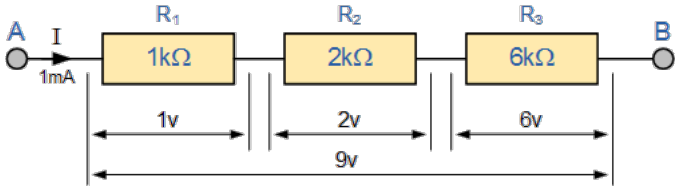
Applications:

- The primary purpose of this circuit is to scale down the input voltage to a lower value based on the ratio of the two resistors.

8. what is the current and voltage output if resistor connect in series and parallel

Connection	Voltage	Current
Series	Voltage divides across resistors	Current remains the same
Parallel	Voltage remains the same	Current divides among branches

 # Series Connection of resistor:
Current is same but voltage get divide across resistors



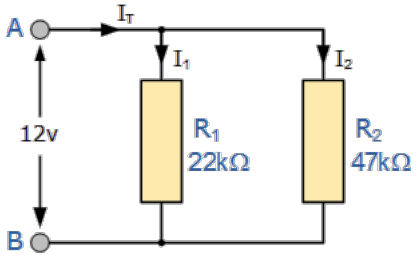
By Ohm's Law V = IxR
Here ,
R = R1 + R2 + R3 = 9Kohm
I = 1mA
V = 9Kohm

1) Voltage across R1 is
V = IxR = 1mA x 1K = 1v

2) Voltage across R2 is
V = IxR = 1mA x 2K = 2v

3) Voltage across R3 is
V = IxR = 1mA x 6K = 6v

 # Parallel Connection of Resistor:
Voltage remain same but current get divide across resistors



By Ohm's Law, **I = V/R**

Here ,
I = I1 + I2 + In

V = 12 ohm

1) Current across R1 is
I = V / R = 12 / 22K = 0.5 mAv

2) Current across R2 is
I = V / R = 12 / 47K = 0.25mA

9. What is Current and Voltage?

- Current (I): Flow of electric charge. Measured in Amperes

  $I = Q / T$

- Voltage (V): Electrical potential difference between two points. Measured in Volts (V).
  Acts like "pressure" pushing electric charge.

10. Ohm's Law and Kirchhoff's Laws

Ohm's Law:

A law that states that the voltage across a resistor is directly proportional to the current flowing through the resistance.

$V=I\times R$

Kirchhoff's Laws:

1. Kirchhoff's Current Law (KCL):

   algebraic sum of all currents entering and exiting a node must equal zero.

   i.e. Total current entering a node = Total current leaving

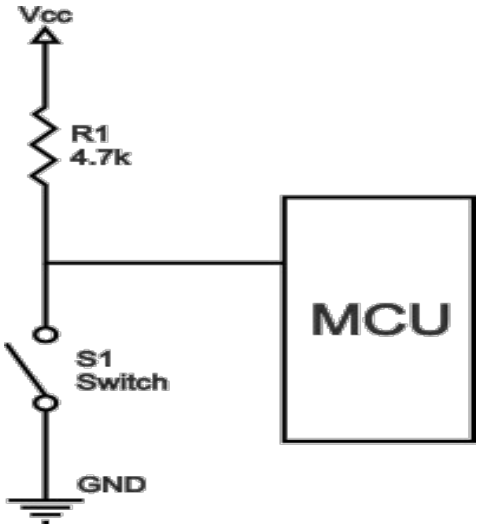
2. Kirchhoff's Voltage Law(KVL):

   Total voltage in a closed loop = 0

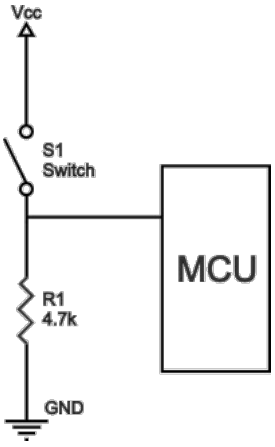
11. Difference Between Pull-up and Pull-down Resistor

Aspect	Pull-up Resistor	Pull-down Resistor
Connection	Connected between input pin and Vcc	Connected between input pin and GND
Default Logic	Keeps pin **HIGH** by default	Keeps pin **LOW** by default
Purpose	Avoids floating input, ensures HIGH when switch open	Avoids floating input, ensures LOW when switch open

#Pull Up



#Pull Down



12. Number Conversion

1. Decimal to Binary: Use successive division by 2.
Example: $10_{10} = 1010_2$

2. Binary to Decimal: Multiply each bit by 2^position.
Example: $1010_2 = 10_{10}$

3. Decimal to Hexadecimal: Use successive division by 16.
Example: $255_{10} = FF_{16}$

4. Hexadecimal to Decimal: Multiply each digit by 16^position.
Example: $FF_{16} = 255_{10}$

13. What is PORT and Setting Bits

A PORT is a group of pins on a microcontroller used for input/output.

To set a bit as output: TRISx = 0 (where x is pin number)
To set a bit as input: TRISx = 1

14. Difference Between if-else and switch-case

Feature	**if-else**	**switch-case**
Condition Type	Checks multiple conditions	Checks a single variable against multiple cases
Data Types	Works with all data types	Works with int, char, enum (not float)
Performance	Slower if many conditions	Faster and cleaner for many cases

15. Enabling and Disabling Internal Pull-up Resistor

In the PIC microcontroller PORTB pins internal pull up, A single control bit can turn ON all the pull Ups

This is performed by clearing 7th bit of **OPTION_REG** i.e. OPTION_REG<7>

16. Function and Function Prototype

A function is a reusable block of code.
Function prototype declares the function before use.
Example:
int add(int a, int b); // Prototype

// Function definition
int add(int a, int b)
{
    return a + b;
}

17. Types of Function Declarations

There are 4 different types

1. Function with no arguments and no return value

```c
#include <stdio.h>

void greet() {
    printf("Hello! Welcome to Embedded C Programming.\n");
}

int main() {
    greet();  // Function call
    return 0;
}
```

2. Function with arguments and no return value

```c
#include <stdio.h>

void displaySum(int a, int b) {
    int sum = a + b;
    printf("Sum = %d\n", sum);
}

int main() {
    displaySum(10, 20);  // Function call with arguments
    return 0;
}
```

3. Function with no arguments and return value

```c
#include <stdio.h>

int getNumber() {
    int num = 42;
    return num;
}

int main() {
    int value = getNumber();  // Function call and return value capture
    printf("Returned value = %d\n", value);
    return 0;
}
```

4. Function with arguments and return value

```c
#include <stdio.h>

int multiply(int x, int y) {
    return x * y;
}

int main() {
    int result = multiply(5, 4);  // Function call with arguments and return value
    printf("Multiplication = %d\n", result);
    return 0;
}
```

18. What is Array and Index Value

Array is a collection of similar data types.
Index starts from 0.
Example: int arr[5] = {1, 2, 3, 4, 5}; // here arr[0] = 1

19. Runtime and Compile Time Array + String Example

Compile Time Array: Defined at coding time.

Run Time Array: Defined at runtime using dynamic memory.
Pushing elements in array in run time.

20. What is for loop and Difference with while loop

Aspect	for loop	while loop
Structure	for(init; condition; inc)	init; while(condition) { inc; }
Use case	Known iteration count	Unknown iteration count
Initialization	Inside loop	Outside loop

21. LCD Pin Diagram

LCD (16x2) has 16 pins:

Pin	Description
1 (VSS)	Ground
2 (VDD)	Power supply (+5V)
3 (V0)	Contrast control
4 (RS)	Register Select
5 (RW)	Read/Write
6 (EN)	Enable
7-14 (D0-D7)	Data pins
15 (LED+)	Backlight +ve
16 (LED-)	Backlight -ve

22. Sending Command to LCD (lcd_command function)

Steps inside lcd_command():

- 1. RS = 0 (Command Register)
- 2. RW = 0 (Write operation)
- 3. Put command on data bus
- 4. EN = 1 -> 0 (Enable pulse to latch data)
- 5. Add delay for processing command

```
// Function to send commands to the LCD
void LcdCommand(uint8_t i)
{
    PORTC &= ~(0x1 << 3);  // Set RS (RC3) = 0 (indicates command mode)
    PORTD = i;             // Place     command on PORTD
    PORTC |= (0x1 << 0);   // Set EN (RC0) = 1 (enable pulse start)
    __delay_ms(100);       // Small delay for command execution
   PORTC &= ~(0x1 << 0);  // Set EN (RC0) = 0 (enable pulse end)
}
```

23. Sending Data to LCD (lcd_data function)

Steps inside lcd_data():

- 1. RS = 1 (Data Register)
- 2. RW = 0 (Write operation)
- 3. Put data on data bus
- 4. EN = 1 -> 0 (Enable pulse to latch data)
- 5. Add delay for displaying data

```
// Function to send data (characters) to the LCD
void LcdData(uint8_t i)
{
        PORTC |= (0x1 << 3);       // Set RS (RC3) = 1 (indicates data mode)
        PORTD = i;                 // Place data on PORTD
        PORTC |= (0x1 << 0);       // Set EN (RC0) = 1 (enable pulse start)
        __delay_ms(100);           // Small delay for command execution
        PORTC &= ~(0x1 << 0);   // Set EN (RC0) = 0 (enable pulse end)
}
```

24. How to Clear One Bit Without Changing Others in PORT

Use bitwise AND with NOT:

```
PORTx &= (~(0x1 << bit_position)); // Here  bit_position is from 0 to 7 for 8 Bit registor
```

25. How to Set One Bit Without Changing Others in PORT

Use bitwise OR:

```
PORTx |= (0x1 << bit_position); // Here  bit_position is from 0 to 7 for 8 Bit registor
```

26. what is bitwise operator and shift operator (i will give question at on time)

Bitwise Operations in C

Bitwise operations in C are used to manipulate individual bits of data. They are fast and efficient, often used in embedded systems, and low-level programming.

Bitwise Operators List and Examples

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	|	OR
^	XOR (Exclusive OR)	Sets each bit to 1 if only one is 1
~	NOT (One's Complement)	Inverts all the bits
<<	Left Shift	Shifts bits to the left and fills 0s
>>	Right Shift	Shifts bits to the right

✅ **1. Bitwise AND (&)**

```
#include <stdio.h>

int main() {
    int a = 5;           // 0101
    int b = 3;           // 0011
    int result = a & b;  // 0001 = 1
    printf("a & b = %d\n", result);
    return 0;
}
```

✅ **2. Bitwise OR (|)**

```
#include <stdio.h>

int main() {
    int a = 5;            // 0101
    int b = 3;            // 0011
    int result = a | b;   // 0111 = 7
    printf("a | b = %d\n", result);
    return 0;
}
```

✅ 3. Bitwise XOR (^)

```
#include <stdio.h>

int main() {
    int a = 5;            // 0101
    int b = 3;            // 0011
    int result = a ^ b;   // 0110 = 6
    printf("a ^ b = %d\n", result);
    return 0;
}
```

✅ 4. Bitwise NOT (~)

```
#include <stdio.h>

int main() {
    int a = 5;          // 0000 0101
    int result = ~a;    // 1111 1010(2's complement of 6) = -6
    printf("~a = %d\n", result);
    return 0;
}
```

✅ 5. Left Shift (<<)

```
#include <stdio.h>

int main() {
    int a = 5;              // 0000 0101
    int result = a << 1;    // 0000 1010 = 10
    printf("a << 1 = %d\n", result);
    return 0;
}
```

✅ 6. Right Shift (>>)

```
#include <stdio.h>

int main() {
    int a = 5;              // 0000 0101
    int result = a >> 1;    // 0000 0010 = 2
    printf("a >> 1 = %d\n", result);
    return 0;
```

27. Logic Gates: Truth Table and Diagram

AND Gate:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate:

A	Y
1	0
0	1

XOR Gate:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND Gate:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

NOR Gate:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

28. What is Diode and Its Applications

A diode is a semiconductor device that allows current in only one direction, hence it act as one way switch.
Applications

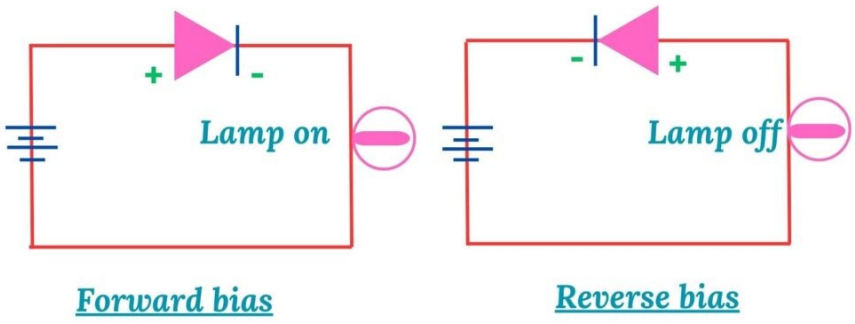
- 1. Rectifiers (AC to DC conversion)
- 2. Clipping and clamping circuits
- 3. Voltage protection (flyback diode)
- 4. Logic gates (in digital circuits)

Forward Bias: Anode connected to +, cathode to - → current flows.
Reverse Bias: Anode connected to -, cathode to + → current blocked.

29. What forward and Reverse bias in diode

Forward Bias:
Anode connected to +, cathode to - → current flows.

Reverse Bias:
Anode connected to -, cathode to + → current blocked.



30. What is Transitor and its types

A transistor is a semiconductor device used for switching and amplification.

Types:

1. Bipolar Junction Transistor (BJT)
   - NPN and PNP
   - Three terminals: Base, Collector, Emitter
   - Works on current control
2. Field Effect Transistor (FET)
   - JFET and MOSFET
   - Works on voltage control

Operation: Small input at base (or gate) controls large current between collector and emitter (or drain and source).

31. Difference between Byte and Bit

Bit: Smallest unit of data, represent by 0 or 1

Byte; 8 Bits

In 8-bit variable: $2^8 = 256$ values (0 to 255)
In 10-bit variable: $2^{10} = 1024$ values (0 to 1023)

32. What is Datatype in C and their byte values

Primary data types

Datatype	Description	Size (Bytes)
char	Character or small integer	1
int	Integer	2 or 4
float	Decimal point number	4
double	Large decimal number	8
short	Smaller integer	2
long	Larger integer	4 or 8

Derived Data Types

Data Type	Description	Example
Array	Collection of elements	`int arr[5];`
Pointer	Stores address of another variable	`int *p = &a;`
Function	Group of statements	`int sum(int a, int b);`
Structure	Group of different data types	`struct student { int id; char name[20]; };`

User Defined Data Types

Keyword	Description	Example
`struct`	Combines different data types	`struct Book { char name[20]; int pages; };`
`union`	Shares memory among members	`union Data { int i; float f; };`
`enum`	Defines named integer constants	`enum color { RED, GREEN, BLUE };`
`typedef`	Creates new data type names	`typedef int age; age a = 25;`

33. What is PWM with Applications

PWM (Pulse Width Modulation) is a technique to control analog signal using digital pulses.
Applications:

Key components of PWM

Duty Cycle: proportion of the ON time to the total period

Duty = (HighTime / TotalTime) x 100 %

Frequency: The rate at which the puls are switched On and OFF

Period

- 1. Motor speed control
- 2. LED brightness control
- 3. Switching power supplies

34. Difference between Time Period and Duty Cycle

Time Period: Time taken for one complete cycle.
Duty Cycle: Percentage of time signal is HIGH in one cycle.
Duty Cycle = (Ton / T) x 100

35. Units of Time and Frequency

Time: Seconds (s), milliseconds (ms), microseconds (μs)
Frequency: Hertz (Hz)
Time = 1 / Frequency
ex. 1Hz = 1 Cycle per second

36. Registers Used in PWM

In PIC16F877A, the PWM uses these registers:
1. CCP1CON, CCP2CON
2. T2CON
3. PR1, PR2
4. CCPR1L, CCPR2L
5. TMR2
   - CCP1CON and CCP2CON is used to control PWM
   - `T2CON` is used to configure and control **Timer2**, which acts as the **time base** for generating PWM signal
   - PR1 and PR2 register is used to specify the PWM period
   - PWM dutycycle is specified by writing to the CCPR1L registor and CCP1CON<5:4> bit
   - CCPR1L contain 8 MSB and CCP1CON<5:4> contain two LSB
   -So total 10 bit value represented by CCPR1L:CCP1CON<5:4>
   - TMR2 is used to steps for generating PWM signal

37. Formula for Duty Cycle and Time Period

```
Pwm duty cycle is specified by writing to CCPR1L and CCP1CON<5:4>
FOSC=6000000
PWM Frequency(Fpwm) = 1khz
TMR2_PRESCALE_VALUE = 16
PR2 = (FOSC/(4*Fpwm*TMR2_PRESCALE_VALUE))-1
CCPR1L:CCP1CON<5:4> = DUTYCYCLE*Fosc/TMR2_PRESCALE_VALUE

Find PR2 value:

PR2 = 6000000/(4*1000*16)-1 =94(0x5E)

Find Dutycycle value:

For 10% duty cycle :
         Dutycycle time = 0.1/pwm_frequency = 0.1/1000
CCPR1L:CCP1CON<5:4> =0.0001*6000000/16 = 38 =>0x26 =>0010 0110
Load CCPR1L = 0000 1001 ; CCP1CON<5:4> = 10

For 50% duty cycle :
         Dutycycle time = 0.5/pwm_frequency = 0.5/1000
CCPR1L:CCP1CON<5:4> =0.0005*6000000/16 = 188 =>0xBC =>1011 1100
Load CCPR1L = 0010 1111 ; CCP1CON<5:4> = 00

For 80% duty cycle :
         Dutycycle time = 0.8/pwm_frequency = 0.8/1000
CCPR1L:CCP1CON<5:4> =0.0008*6000000/16 = 300 =>0x12C =>0001 0010 1100
Load CCPR1L = 0100 1011 ; CCP1CON<5:4> = 00
```

38. What is timer and how much timers in pic16f877a and how that timer using in pwm function

PIC16F877A has 3 Timers:
1. Timer0 (8-bit)
2. Timer1 (16-bit)
3. Timer2 (8-bit - used in PWM)

PWM uses Timer2 for generating signal.

PWM Operation

Ones the PWM is configured and TMR2 is enabled, TMR2 is start incrementing on the prescalar. Ones the TMR2 value equals to CCPR1L:CCP1CON<5:4> the PWM pin pulled to LOW. The timer till continue and it incrementing till it matches with the period PR2. After which the PWM pin will be pulled HIGH and TMR2 is reset to 0.

39. What is ADC with Applications

ADC (Analog to Digital Converter) converts analog voltage to digital data.

Some ADC terms

#Sampling Rate:

The number of samples on ADC take per second from the analog input signal.

#Quantization:

The process of approximating the sampled analog value to the nearest discrete digital level.

#Resolution

The number of discrete levels or bits on the ADC can represent, the accuracy of conversion.

e.x. 8 Bit, 10 Bit etc

for 8 bit value from 0 to 255

for 10 bit values from 0 to 1023

for 12 bit values from 0 to 4095

Applications:

- 1. Temperature sensors
- 2. Light sensors
- 3. Audio signal processing

40. Explain SAR Operation (Successive Approximation Register)

SAR ADC converts analog to digital by binary search:

1. **Start Conversion**:
   - The ADC starts a conversion cycle when triggered.

2. **Hold the Analog Input**:
   - A **sample and hold** circuit captures and freezes the analog input voltage.

3. **SAR Logic Starts Binary Search**:
   - The SAR sets the **most significant bit (MSB)** to 1, others to 0.
   - A **DAC** (Digital-to-Analog Converter) inside the ADC converts this digital value to analog.

4. **Comparator Compares**:
   - The input analog voltage is compared with the DAC output.
   - If the input is **higher**, the bit is kept as 1.
   - If the input is **lower**, the bit is cleared to 0.

5. **Repeat**:
   - This process continues **bit-by-bit**, from MSB to LSB, until all bits are decided.

6. **Output the Result**:

- The SAR now holds the digital equivalent of the analog input.

41. What is the ADC resolution and how its impact accuracy of ADC output

ADC resolution refers to the number of distinct digital values that on ADC convertor can produce to represent an analog input signal.

Definition:
The resolution of an ADC is the number of bit used in the digital representation of the analog input.
N bit ADC, the total number of discrete digital levels or steps size 2^N
ex. 8 bit ADC → 2^8 → 256 levels

42. What are the types of registors using in ADC

 # ADCON0

To control the ADC operation and select the input channel
It having
ADON<bit0> : To Enable ADC
Go/Done<bit2> : To start ADC conversion
CHS<bit5:bit3> : To select analog channel

#ADCON1

Configure the reference voltage select the ADC pin configuration

#ADRESL

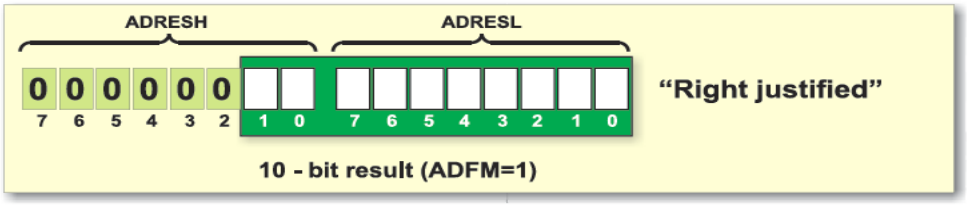
The registor which store the low byte of the 10 bit ADC conversion result

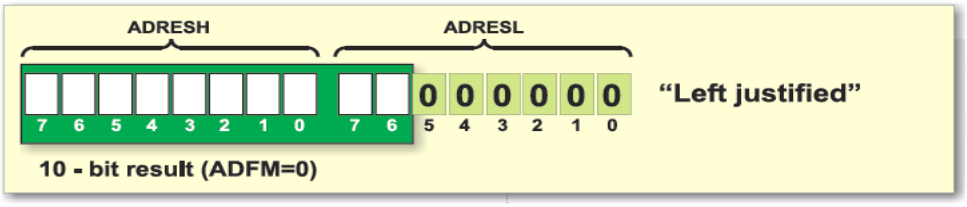
#ADRESH

The registor hold the high byte of the 10 bit ADC conversion result

43. What is the difference between right justify and left justify in ADC

Justification of the ADC refer to how the 10 bit result of the ADC conversion is aligned within the ADRESH and ADRESL reg





 # Right Justify

8 LSB of 10 bit result placed in ADRESL

2 MSB of 10 bit result placed in ADRESH

 # Left Justify

8 MSB of 10 bit result placed in ADRESH

2 LSB of 10 bit result placed in ADRESL

44. How to store 10 bit PWM duty cycle value in registers

PWM duty cycle register are CCPR1L and CCP1CON

8 MSB which are stored in CCPR1L

2 LSB which are stored in CCP1CON<5:4>

45. What Is Program Counter And Working Of The Program Counter

Program counter is the special purpose register in a micro-controller, which hold the memory address of the next instruction to be executed.

Program Counter (register) helps to CPU for Fetch-Decode-Execute the instruction

Working....

1. The CPU reads the memory address stored in the Program Counter (PC).
2. The fetched instruction is decoded to determine what operation it represents by CPU
3. The CPU executes the instruction
4. Program counter is incremented to point to the next instruction

Note:: above 4 steps repeated continuously

46. What Is The Bus In Micro-Controller

A bus is essentially a collection of wires or communication lines that allow information(data, address, control signal) to be shared components of the system.

Bus can be a full duplex or half duplex mode

47. We Connect Temperature Sensor To Adc Pin And Assume That Sensor Give 3 Volt To Adc So What Is Digital Value We Get ? And Calibrate That Digital Value To 0 To 100

Assume Voltage range of micro-controller is 0 to 5V

for the 10 Bit ADC

0v ---→ 0

5v ----→ 1023

so for 3 volt

3v ----→ 614

Now Calibration for 0 to 100

0 ------→ 0

1023 -----→ 100

so for 614

614 ----→ 60

60 is the calibrated value for 3 volt for the range of 0 to 100

48. What Is Go_Done Bit What Is The Use Of That Bit In Adc

Go-Done bit in a ADCON0 register that indicate the status of ADC conversion processing

Purpose:

It control the start of an ADC conversion and provide information and whether conversion is complete.

To start the ADC conversion Go-Done bit should HIGH

When ADC conversion complete then Go-Done bit becomes 0 automatically

49. Give The 5 Best Embedded Company Names And Their Embedded Product And Application Of That Product

50. What Is Iot(Internet Of Things) And Explain Working Principle Of Atm Machine And Traffic Light And Washing Machine And How To You Program These.